Technical Design Document: Bananaman 3D Side-Scroller Vertical Slice

1. Overview

Project Name: Bananaman 3D Side-Scroller
Genre: 2.5D platformer
Target Platform: PC (Unity .NET Framework 4.7.1, C# 9.0)
Scope: Vertical slice demonstrating core mechanics, PSX-inspired visuals, spline-based traversal, collectibles and retro UI, hazards, and check-pointing.

Key Pillars:
Event-driven architecture for decoupled systems
Spline-guided player movement (Catmull-Rom)
Retro PSX post-processing
Modular hazard/collectible/checkpoint systems

---

2. Architecture

2.1 Event Hub Pattern
Central dispatcher: EventHub singleton publishes/subscribes to typed events.
Benefits: No direct coupling between gameplay systems; UI, audio, VFX react to game state changes without modifying core logic.

Key Events:
TeleportEvent, ScreenFadeEvent (teleport sequencing)
CollectItemEvent, CollectItemApprovalEvent (approval-gated pickup)
HealthChangeEvent, GameOverEvent (player lifecycle)
PlatformRideEvent (platform mounting)
//RunnerSegmentStartEvent, RunnerSegmentEndEvent (runner mode toggle) // In progress
CheckpointReachedEvent, RespawnRequestEvent (checkpointing)
LevelFinishedEvent (level completion)
Level Transitions
//Cinematic cutscene(in progress)

```
// Publish: systems listen without tight coupling
EventHub.Instance.Publish(new LevelFinishedEvent(0));

// Example subscriber
void OnEnable() { EventHub.Instance.Subscribe<ScreenFadeEvent>(OnFade); }
void OnFade(ScreenFadeEvent e) { canvasGroup.alpha = e.FadeIn ? 1f : 0f; }
```

```

2.2 Manager Layer
LevelController: Scene lifecycle (start, finish, game over)
EnemyManager: Enemy registry, wave spawning
CheckpointManager: Respawn points, state persistence
TeleportManager: Coroutine-driven fade/teleport/fade-in
CameraController: Free/chase mode switching, FOV/orientation

2.3 View Layer
ManagerView: Orchestrates canvas enable/disable
CollectableBar: Dynamic icon instantiation per resource type
CollectablePopup: Transient pickup feedback
HubGate: Level unlock UI

---

**3. Player Systems**

3.1 Movement & Physics
Rigidbody-driven: Continuous dynamics, grounded checks via contact counts
Slope handling: Project movement onto surface normal; uphill/downhill speed factors; climb-assist force
Coyote time + jump buffer: Frame-grace for jumps near ledge/ground transitions
Platform riding: Strong-lock MovePosition on moving platforms; offset capture to prevent drift

```
// Strong-lock movement on platform (drift-free)
Vector3 platformPos = m_currentPlatform.transform.position;
Vector3 targetWorld = platformPos + m_platformOffsetPos;
targetWorld.y = transform.position.y;
m_rigidbody.MovePosition(targetWorld);

```

3.2 Spline-Guided Traversal
PathAgent: Samples Catmull-Rom spline (CatmullRomGenPoints)
Distance-based positioning: Player snapped to nearest path point each frame (XZ only, Y free)
Suspension mode: Teleport temporarily disables snap; preserves XZ offset to avoid jarring repositioning
Runner mode: RunnerSegmentController auto-advances along path with discrete lane switching

```
// Sample Catmull-Rom path by distance
float d = t * path.GetPathLength();
Vector3 pos = path.GetPosByDist(d);
```
```

```
transform.position = pos;

// Agent-guided follow (stable direction)
Vector3 fwd = m_pathAgent.GetCurrentDirection();
Vector3 here = m_pathAgent.GetCurrentPosition();
```

3.3 Combat & Ranged Attack

Melee: Punch coroutine with windUp/active/recover phases; weapon colliders enabled only during hit window

Ranged: Unlockable via collectibles; spawns Bullet with homing, damage, visual scale config

```
// Fire banana projectile with safe physics init
Vector3 chestBase = transform.position + Vector3.up * m_rangedChestHeight;
Vector3 dir = m_animationController.GetFacingDirection().normalized;
Bullet bullet = Instantiate(m_playerBulletPrefab, chestBase,
Quaternion.LookRotation(dir));

bullet.ConfigureTargets(damagePlayer: false, damageEnemies: true);
bullet.SetDamage(m_rangedDamage);
bullet.SetSpeed(m_rangedSpeed);
bullet.SetVisualScale(m_rangedBulletScale);
bullet.InitPhysics(GetComponentsInChildren<Collider>(true));
bullet.ApplyMovement(dir);
```

3.4 Teleportation

Ground snap: SphereCast probes below teleport point; rejects steep surfaces or excessive drop

Path realignment: PathAgent.FindPositionOnPath3D() uses full 3D distance to avoid snapping to distant segments

Suspension: Snap suspended for m_pathSnapSuspendDuration post-teleport to prevent immediate pull-back

```
// Sequenced fade -> teleport -> fade-in
IEnumerator TeleportCoroutine(PlayerController player, Transform dest)
{
   EventHub.Instance.Publish(new ScreenFadeEvent(true));
   yield return new WaitForSeconds(3f);
   EventHub.Instance.Publish(new TeleportEvent(dest.position));
   EventHub.Instance.Publish(new ScreenFadeEvent(false));
}
```

```
// Player-side approval gate
void CollectItemApprovalEvent(CollectItemApprovalEvent e)
{
   var c = e.CollectableItem.Collectable;
   if (m_healthCurrent < m_healthMax && c.CollectabeType == CollectabeType.Health)
   {
      e.Approve();       // single-shot approval
      Heal(c.Count);     // apply benefit
   }
}

// Core approval
public void Approve()
{
   if (!m_isApproved) { m_callback(m_collectableItem); m_isApproved = true; }
}
```

---

4. Collectibles & Progression

4.1 Approval-Gated System
Two-phase pickup: Trigger publishes CollectItemApprovalEvent → player logic calls Approve() → callback fires CollectItemEvent
Race-safe: m_isApproved flag prevents double-collection

```
// Player-side approval gate
void CollectItemApprovalEvent(CollectItemApprovalEvent e)
{
   var c = e.CollectableItem.Collectable;
   if (m_healthCurrent < m_healthMax && c.CollectabeType == CollectabeType.Health)
   {
      e.Approve();       // single-shot approval
      Heal(c.Count);     // apply benefit
   }
}

// Core approval
public void Approve()
{
   if (!m_isApproved) { m_callback(m_collectableItem); m_isApproved = true; }
}
```

4.2 Dynamic UI Rendering
CollectableBar: Instantiates icons per count; offsets per-icon + global

```
// Per-icon + global offsets
for (int i = 0; i < count; i++)
{
    var icon = Instantiate(m_iconPrefab, m_container);
    var local = new Vector3(m_offsetItem * i, 0f, 0f);
    icon.transform.localPosition = m_offsetGlobal + local;
}
```

---

5. Hazards & Obstacles

5.1 Kill Volumes
KillYThreshold: Y-plane or BoxCollider volume; armed after delay + input unlock; downward motion gate; grace period
HazardVolume: Damage-on-contact; configurable per-type

5.2 Boulder Spawner
Timed/pooled: Coroutine loop spawns RollingHazard at intervals
Path-guided or free-fall: Hazards follow spline or physics

```
// Timed spawn loop
IEnumerator SpawnLoop()
{
    while (true)
    {
        SpawnBoulder();
        yield return new WaitForSeconds(m_spawnInterval);
    }
}

void SpawnBoulder()
{
    var b = Instantiate(m_boulderPrefab, m_spawnPoint.position, m_spawnPoint.rotation);
    // optional: apply initial impulse/roll
    // b.GetComponent<Rigidbody>()?.AddForce(m_spawnPoint.forward *
m_launchForce, ForceMode.Impulse);
}
```

5.3 Moving Platforms
MovedPlatform: Linear/rotational motion; publishes PlatformRideEvent on player contact
PlatformRidePathHandler: Listens to event; aligns platform to path velocity

```
// Player mounts/dismounts platform cleanly
void OnPlatformRide(PlatformRideEvent e)
{
    if (e.Player != this) return;
    m_ridingPlatform = e.IsMount;
    if (!m_ridingPlatform) m_hasRideOffset = false;
}
```

---

6. Checkpointing & Respawn

6.1 Flow
Player enters CheckpointTrigger → publishes CheckpointReachedEvent
CheckpointManager stores position + index
On death → RespawnRequestEvent → manager teleports player to last checkpoint
Optional: CheckpointAutoPlacer distributes checkpoints along spline at intervals

6.2 State Persistence
Checkpoint index stored in CheckpointManager; respawn resets player health/state via Initialize()

---

7. Camera Systems

7.1 Base CameraController
Free mode: Lerped follow with configurable offset; optional flip for left/right path directions
Chase mode: Runner segments activate CameraChaseModeHandler

7.2 Runner Chase Rig
Event-driven activation: Subscribes to RunnerSegmentStartEvent/EndEvent
Path-aligned offset: Computes right/up/forward frame from PathAgent.GetCurrentDirection(); applies offset in local space
Smooth follow/look: Exponential lerp for position/rotation

```
// Chase camera positioning
Vector3 forward = m_targetPath.GetCurrentDirection();
forward.y = 0f; forward.Normalize();
Vector3 right = Vector3.Cross(Vector3.up, -forward).normalized;
Vector3 worldOffset = right * m_localOffset.x + Vector3.up * m_localOffset.y + (-forward) * Mathf.Abs(m_localOffset.z);
Vector3 targetPos = m_target.position + worldOffset;
_cam.transform.position = Vector3.Lerp(_cam.transform.position, targetPos, 1f - Mathf.Exp(-m_followLerp * Time.deltaTime));
```

```

---

**8. Runner Segments**

8.1 Auto-Runner Logic (RunnerSegmentController)
Auto-advance: PathAgent.MoveForward(m_runSpeed * Time.fixedDeltaTime)
Lane switching: Discrete X offsets; input switches lane index; smooth lerp to target offset
Acceleration: Optional speed ramp over time
Input lock: Publishes PlayerInputLockEvent to disable free movement

```
// Lane switch + position update
m_pathAgent.MoveForward(m_runSpeed * Time.fixedDeltaTime);
_laneOffsetCurrent = Mathf.Lerp(_laneOffsetCurrent, _laneOffsetTarget, 1f -
Mathf.Exp(-m_laneSwitchLerp * Time.fixedDeltaTime));
Vector3 target = m_pathAgent.GetCurrentPositionWithXOffset(_laneOffsetCurrent);
target.y = m_player.transform.position.y;
m_player.SetPosition(target);
```

8.2 Timer (RunnerTimer)
Duration-based: Subscribes to RunnerSegmentStartEvent; counts down; publishes EndEvent on expiry

---

**9. Visual Systems**

9.1 PSX Post-Processing
PSXPostProcess: OnRenderImage fullscreen blit with color quantization + dither
Shader: Hidden/PSX/ColorDither implements Bayer matrix dithering
Tunables: Palette steps (4–256), dither enable, intensity

```

// Fullscreen PSX dither/post
void OnRenderImage(RenderTexture src, RenderTexture dst)
{
    m_ditherMaterial.SetFloat(ID_ColorSteps, Mathf.Max(4, m_paletteSteps));
    m_ditherMaterial.SetFloat(ID_EnableDither, m_enableDither ? 1f : 0f);
    m_ditherMaterial.SetFloat(ID_DitherIntensity, Mathf.Clamp01(m_ditherIntensity));
    Graphics.Blit(src, dst, m_ditherMaterial);
}
```

9.2 Vertex Snapping
PSXUnlitVertexSnap: Vertex shader quantizes positions to grid for polygon-jitter effect

9.3 Retro Render Scale
```

RetroRenderScale: Downsamples camera to low-res target, blits to screen with point filtering

9.4 Damage Flash
DamageFlashEffect: Per-renderer material color/emission flash on damage; coroutine-driven restore

9.5 Grain Overlay
CanvasGrainOverlay: Additive UI layer with noise texture; lerped alpha for film grain

---

10. Decoration & Ambiance

10.1 Background Decorator
BackgroundDecorator: Spawns distant parallax layers at configurable intervals along spline

10.2 Path Decorator
PathDecorator: Distributes ground clutter (rocks, foliage) along path segments

10.3 Parallax Layers
ParallaxLayer: Scrolls based on camera X offset; multi-depth for pseudo-3D

10.4 Biome System
BiomeDecoratorPresets: Asset-driven biome configs (terrain color, fog, skybox, decoration density)
DecorationRuntime: Swaps biome assets per level/zone

---

11. Level Flow

11.1 Boot Sequence
Loader initializes ServiceProvider (saves, internet, accounts)
LevelController.Awake() finds PlayerController, EnemyManager, CameraController
CheckpointManager registers triggers
Player spawns; PathAgent.FindPositionOnPath() snaps to spline

11.2 Gameplay Loop
Player navigates spline; collectibles/hazards react via events
Damage/death → respawn at checkpoint
Reach FinishTrigger (proximity or trigger) → LevelFinishedEvent → LevelEndCinematic → FinishMenu

11.3 Level Load Request
LevelLoadRequest: Static state machine; Set(group, levelNum) → Loader transitions scenes

---

12. Data Flow Diagrams

12.1 Collectible Pickup

```
Player OnTrigger → CollectItemApprovalEvent
            ↓
PlayerController.CollectItemApprovalEvent()
   ↓ (conditional)
   e.Approve()
      ↓
      Callback → CollectItemEvent
         ↓
         UI (CollectableBar, Popup)
         Player (Heal, Unlock)
```

12.2 Teleport Sequence

```
TeleportTrigger.OnTrigger → TeleportManager.OnTeleport()
   ↓
   Coroutine: ScreenFadeEvent(true)
      ↓ WaitForSeconds(3f)
      TeleportEvent(dest)
      ↓
      ScreenFadeEvent(false)
   ↓
PlayerController.OnTeleport() → SetPosition() → Ground snap + path realignment
```

12.3 Runner Segment

```
RunnerSegmentTrigger.OnTriggerEnter → RunnerSegmentStartEvent
   ↓
   RunnerSegmentController.OnRunnerStart() → StartRunner()
   CameraChaseModeHandler.OnRunnerStart() → _active = true; FOV change
   RunnerTimer.OnRunnerStart() → countdown
      ↓ (duration expires or exit)
      RunnerSegmentEndEvent
         ↓
         All subscribers restore normal state
```

---

13. Technical Constraints & Optimizations

13.1 Physics

Continuous Dynamic: Player uses ContinuousDynamic to prevent tunneling

Trigger optimization: Hazards/collectibles use QueryTriggerInteraction.Collide; ignore volume triggers for ground checks

13.2 Spline Caching

PathAgent.BuildCache(): Pre-samples spline at fixed intervals for fast distance-to-position lookups

Spatial partitioning (optional): Accelerates nearest-point queries

13.3 Event Pooling

Events are struct-based (no GC alloc); EventHub uses Dictionary<Type, Delegate> for O(1) dispatch

13.4 UI Instancing

Object pooling: CollectablePopup could pool instances; current implementation uses Destroy() after fade

Canvas batching: All UI under one Canvas; static elements cached

14.Testing & Debug Tools

14.1 Debug Modes

PlayerController.m_debugPhysics: Logs grounding, platform contacts, path snap state

Bullet.m_debugMode: Logs spawn, collider config, hit detection

FinishTrigger: Always logs trigger/proximity events; layer collision state

PathAgent.OnDrawGizmosSelected(): Visualizes spline, current position, direction

14.2 Cheat/Test Hooks

m_alwaysAllowRanged: Bypasses collectible unlock for ranged attack

DebugRunnerStarter: Auto-fires runner segment on Start for quick iteration

LevelLoadRequest.Set(): Direct scene transition from inspector/console

15. Future Extensions

15.1 Split screen Co-op Multiplayer

Approval-gated collectibles already race-safe; extend with client-authority validation

15.2 Procedural Levels

PathDecorator + BiomeDecoratorPresets can be fed procedural spline data

15.3 Boss Encounters

Extend event system with BossPhaseEvent, BossDefeatEvent; camera/movement systems already event-driven

15.4 Save System

CheckpointManager state + CollectableModel → ISavesService persistence

16. Conclusion

This vertical slice demonstrates:

Scalable event-driven architecture: Easy to add UI, audio, VFX without touching core logic.
Spline-guided traversal: Cinematic, designer-friendly movement with minimal code
Modular hazard/collectible systems: Drop-in prefabs + event wiring
Variety of Moving platforms with one way colliders and on touch activation.
Retro visual fidelity: PSX shader pipeline for authentic low-poly aesthetic
Robust player physics: Slope handling, platform riding, teleport safety

Next Steps/TO DO:
Fix player animation bugs/smoothness
Fix bugs in end scene cinamatic camera panning/transition
Fix Jitter on some moving platforms
Fix/implement Path switching abilities
Expand enemy variety (ranged, flying)
Implement save/load for progression
Fix WebGL build
Add audio event listeners (footsteps, ambiance, SFX)
Polish checkpointing (particle FX, SFX on reach)
Build level 2–3 to validate biome system